DOCUMENTO TÉCNICO: Explicación Semántica de script.js

Proyecto: Formulario de Registro con Validación Avanzada

Autor: Dr. Héctor Arciniega Valencia

Versión: 1.0

Fecha: Marzo 2026

ÍNDICE DE CONTENIDOS

1. Estructura General y Evento de Inicialización
2. Selección de Elementos del DOM
3. Configuración de Datos (Objetos y Arrays)
4. Sistema de Validación en Tiempo Real (Eventos input)
5. Lógica de Máscara Telefónica Dinámica
6. Simulación Asíncrona de Servidor (Promises)
7. Validación Final y Envío del Formulario
8. Funciones Auxiliares de UI
9. Referencia Rápida de Tokens JavaScript

1. ESTRUCTURA GENERAL Y EVENTO DE INICIALIZACIÓN

Código Base

```
document.addEventListener('DOMContentLoaded', () => { ... });
```

Explicación Detallada

Token/Sintaxis	Categoría	Función Semántica
document	Objeto Global	Representa el árbol completo del DOM (Document Object Model). Es la puerta de entrada para manipular la página.
.	Operador de Miembro	Permite acceder a propiedades y métodos dentro del objeto document.
addEventListener	Método Nativo	Registra una función "oyente" que se ejecutará automáticamente cuando ocurra un evento específico.
DOMContentLoaded'	String Literal	Nombre del evento. Se dispara cuando el HTML ha sido completamente parseado (convertido a un lenguaje entendible para la computadora), **sin esperar** imágenes o CSS externos.
,	Separador	Divide los argumentos de la función: primero el tipo de evento, luego la función a ejecutar.
() => {}	Arrow Function	Sintaxis ES6 para funciones anónimas. Equivale a function() {} pero con this léxico.
{ ... }	Bloque de Código	Delimita el ámbito (scope) donde se ejecutará toda la lógica del script.
;	Terminador	Marca el fin formal de la instrucción (opcional en JS moderno, pero recomendado por claridad).

Propósito: Garantizar que el JavaScript solo intente seleccionar elementos HTML **después** de que estos existan en el DOM, evitando errores de referencia nula.

2. SELECCIÓN DE ELEMENTOS DEL DOM

Patrón Repetitivo

```
const nombreVariable = document.getElementById('id-en-html');
```

Desglose Token por Token

Token	Explicación Técnica
const	Declara una constante de **referencia inmutable**. No puedes reasignar la variable (nombreVariable = otroValor daría error), pero sí puedes modificar las propiedades del objeto referenciado (ej: nombreVariable.value = 'nuevo').
nombreVariable	Identificador. Sigue convención camelCase. Debe ser descriptivo para mantener legibilidad.
=	Operador de asignación. Evalúa la expresión derecha y almacena el resultado en la variable izquierda.
document.getElementById()	Método de búsqueda. Recorre el DOM buscando un elemento cuyo atributo id coincida exactamente con el string proporcionado. Retorna el elemento HTML o null si no lo encuentra.
id-en-html'	Argumento string. Debe coincidir **exactamente** (case-sensitive) con el atributo id definido en el HTML.

Elementos Seleccionados en este Script

```
// Formulario y controles principales
const formulario = document.getElementById('formularioRegistro');  // <form>
const btnEnviar = document.getElementById('btn-enviar');           // <button>
const mensajeExito = document.getElementById('mensaje-exito');     // <div> de feedback

// Inputs Personales
const inputNombre = document.getElementById('nombre');
const inputEdad = document.getElementById('edad');

// Inputs Contacto
const inputEmail = document.getElementById('email');
const inputConfirmarEmail = document.getElementById('confirmar-email');
const selectPais = document.getElementById('pais');                // <select>
const inputTelefono = document.getElementById('telefono');
const spanPrefijo = document.getElementById('prefijo-telefono');    // <span> visual

// Inputs Domicilio (6 campos)
const inputCalle = document.getElementById('calle');
const inputNumero = document.getElementById('numero');
const inputCP = document.getElementById('cp');
const inputColonia = document.getElementById('colonia');
const inputCiudad = document.getElementById('ciudad');
const inputEstado = document.getElementById('estado');
```

```
// Inputs Seguridad
const inputPassword = document.getElementById('password');
const inputConfirmarPassword = document.getElementById('confirmar-password');
```

Nota de Buenas Prácticas: Guardar las referencias en variables const al inicio evita buscar repetidamente en el DOM (operación costosa) cada vez que necesites acceder a un elemento.

3. CONFIGURACIÓN DE DATOS: OBJETOS Y ARRAYS

Objeto Literal configPaises

```
const configPaises = {
    'MX': { prefijo: '+52', mascara: '(XXX) XXX XXXX', longitud: 10, flag: '🇲🇽' },
    'US': { prefijo: '+1', mascara: 'XXX XXX XXXX', longitud: 10, flag: '🇺🇸' },
    // ... más países
};
```

Estructura Semántica

Componente	Explicación
{ ... }	Sintaxis de **objeto literal**. Colección de pares clave-valor.
MX'	**Clave** (key). String que actúa como índice único para acceder a la configuración.
:	Separador clave-valor dentro del objeto.
{ prefijo: ... }	**Valor** (value). En este caso, otro objeto anidado con propiedades específicas.
prefijo, mascara, etc.	Propiedades del objeto interno. Accesibles vía configPaises.MX.prefijo.
'+52', '(XXX) XXX XXXX'	Valores string literales.
10	Valor numérico entero (sin comillas).
'🇲🇽'	String con emoji Unicode. JS trata emojis como caracteres string normales.
,	Separador entre propiedades o entre elementos del objeto principal.

Acceso Dinámico (Uso Posterior)

```
const config = configPaises[paisCodigo];
```

- paisCodigo es una variable con valor 'MX', 'US', etc.
- La notación de corchetes [] permite acceso **dinámico** (el nombre de la propiedad se evalúa en tiempo de ejecución).
- Equivale a: configPaises.MX si paisCodigo === 'MX'.

Array correosRegistrados

const correosRegistrados = ['admin@empresa.com', 'usuario@prueba.com'];

- [...]: Sintaxis de **array literal** (lista ordenada).
- Cada elemento es un string separado por comas.
- Usado para simular una base de datos de correos ya existentes.

4. SISTEMA DE VALIDACIÓN EN TIEMPO REAL (EVENTOS input)

Evento Base para un Input

inputNombre.addEventListener('input', (e) => { ... });

Token	Explicación
input'	Tipo de evento. Se dispara **inmediatamente** cuando el valor del campo cambia (tecla, pegado, dictado por voz, autocompletado). Más confiable que keypress o keyup.
(e)	Parámetro de la función callback. e es el objeto Event que contiene metadata sobre el evento disparado.
e.target	Propiedad del evento. Referencia directa al elemento HTML que originó el event
e.target.value	Propiedad del elemento input. Contiene el texto actual escrito por el usuario.

Expresiones Regulares (Regex) para Validación

Regex para Solo Letras (Nombre)

const regex = /[^A-ZÁÉÍÓÚÑ\s]/gi;

Componente Regex	Significado
/ ... /	Delimitadores de literal RegExp en JavaScript.
[^ ...]	**Clase de caracteres negada**. Coincide con cualquier carácter que **NO** esté listado dentro.
A-Z	Rango: todas las letras mayúsculas del alfabeto inglés.
ÁÉÍÓÚÑ	Literales: vocales acentuadas y eñe mayúscula (soporte español).
\s	**Escape sequence**: representa cualquier espacio en blanco (space, tab, newline).
g	**Flag global**: aplica la búsqueda a TODAS las coincidencias en el string, no se detiene en la primera.
i	**Flag ignore case**: hace la búsqueda insensible a mayúsculas/minúsculas (aunque luego convertimos a mayúsculas manualmente).

Regex para Solo Números

```
const regex = /[^0-9]/g;
```

- [^0-9]: Niega el rango de dígitos del 0 al 9. Coincide con letras, símbolos, espacios, etc.

Regex para Texto con Números (Domicilio)

```
const regex = /[^A-Z0-9\sÁÉÍÓÚÑ]/gi;
```

- Permite letras, números y espacios. Rechaza símbolos especiales como @, #, $, etc.

Método replace() y Encadenamiento

```
e.target.value = e.target.value.replace(regex, '').toUpperCase();
```

Método	Propósito
.replace(regex, '')	Busca todas las coincidencias del regex y las reemplaza con string vacío (efecto: **eliminar**).
.toUpperCase()	Convierte toda la cadena resultante a mayúsculas. Se ejecuta **después** del replace (encadenamiento de métodos).
=	Reasigna el valor procesado de vuelta al input, actualizando lo que ve el usuario.

Normalización de Emails a Minúsculas

```
inputEmail.addEventListener('input', (e) => {
  e.target.value = e.target.value.toLowerCase();
});
```

- toLowerCase(): Método de String que retorna una nueva cadena con todos los caracteres en minúsculas.
- **Justificación:** Los correos electrónicos son case-insensitive por estándar RFC. Usuario@Correo.COM es idéntico a usuario@correo.com. Normalizar previene duplicados falsos.

Iteración con forEach para Múltiples Inputs

```
[inputEdad, inputNumero, inputCP].forEach(input => {
  input.addEventListener('input', (e) => { ... });
});
```

Token	Explicación
[a, b, c]	Crea un array temporal con las referencias a los inputs.
.forEach()	Método de Array. Ejecuta la función proporcionada una vez por cada elemento.
input => {...}	Función flecha. input es la variable local que representa el elemento actual en la iteración.
`error-${input.id}`	**Template literal**. Interpola el valor de input.id dentro del string. Ej: si input.id === 'edad', resulta 'error-edad'.

5. LÓGICA DE MÁSCARA TELEFÓNICA DINÁMICA

Evento change en Select de País

```
selectPais.addEventListener('change', (e) => {
    const paisCodigo = e.target.value;
    const config = configPaises[paisCodigo];

    if (config) {
        // Configurar UI según país seleccionado
    } else {
        // Resetear UI si no hay selección válida
    }
});
```

Concepto	Explicación
change'	Evento específico de elementos <select>. Se dispara cuando el usuario selecciona una opción diferente.
e.target.value	En un <select>, retorna el atributo value de la <option> seleccionada (ej: 'MX').
if (config)	Condición de verdad. Si configPaises[paisCodigo] retorna undefined (país no configurado), el bloque if no se ejecuta.
inputTelefono.disabled = false	Propiedad booleana del elemento HTML. false habilita el campo para escritura.
.placeholder	Texto de ayuda que se muestra cuando el input está vacío.
.textContent	Propiedad para leer/escribir el contenido textual de un elemento (más seguro que innerHTML para texto plano).
.focus()	Método que pone el cursor de texto (foco) en el input, mejorando la experiencia de usuario.

Algoritmo de Enmascaramiento (Core)

```
inputTelefono.addEventListener('input', (e) => {
    // 1. Obtener configuración del país
    const paisCodigo = selectPais.value;
    if (!paisCodigo || !configPaises[paisCodigo]) return;
    const config = configPaises[paisCodigo];

    // 2. Extraer solo dígitos del valor actual
    let valor = e.target.value.replace(/[^0-9]/g, '');

    // 3. Limitar longitud máxima
    if (valor.length > config.longitud) {
        valor = valor.substring(0, config.longitud);
    }

    // 4. Construir valor formateado
    let valorEnmascarado = '';
    let indiceValor = 0;
```

```
    for (let i = 0; i < config.mascara.length; i++) {
        if (config.mascara[i] === 'X' && indiceValor < valor.length) {
            valorEnmascarado += valor[indiceValor];
            indiceValor++;
        } else if (config.mascara[i] !== 'X') {
            valorEnmascarado += config.mascara[i];
        }
    }

    // 5. Actualizar el input
    e.target.value = valorEnmascarado;
});
```

Paso a Paso del Algoritmo

Paso 1: Validación Temprana (Guard Clause)

```
if (!paisCodigo || !configPaises[paisCodigo]) return;
```

- !: Operador NOT lógico. Invierte booleano.
- ||: Operador OR. Si **cualquiera** de las condiciones es verdadera, ejecuta return.
- return;: Termina la función inmediatamente. Evita errores al intentar acceder a propiedades de undefined.

Paso 2: Limpieza de Entrada

```
let valor = e.target.value.replace(/[^0-9]/g, '');
```

- Elimina paréntesis, espacios, guiones, letras, dejando solo dígitos puros para procesamiento.

Paso 3: Control de Longitud

```
if (valor.length > config.longitud) {
    valor = valor.substring(0, config.longitud);
}
```

- .length: Propiedad de String que retorna número de caracteres.
- .substring(inicio, fin): Extrae porción del string. substring(0, 10) toma caracteres del índice 0 al 9 (10 dígitos).

Paso 4: Bucle de Enmascaramiento

```
for (let i = 0; i < config.mascara.length; i++) { ... }
```

- Itera carácter por carácter sobre la **plantilla de máscara** (ej: '(XXX) XXX XXXX').

Dentro del Bucle:

```
if (config.mascara[i] === 'X' && indiceValor < valor.length) {
    valorEnmascarado += valor[indiceValor];
    indiceValor++;
```

```
}
```

- Si el carácter de la máscara es 'X' (marcador de dígito) **Y** aún hay dígitos del usuario por colocar:
  - valor[indiceValor]: Accede al dígito específico del usuario.
  - +=: Concatena ese dígito al resultado.
  - indiceValor++: Avanza al siguiente dígito del usuario.

```
else if (config.mascara[i] !== 'X') {
   valorEnmascarado += config.mascara[i];
}
```

- Si el carácter de la máscara **NO** es 'X' (ej: '(', ')', ' '):
  - Copia ese carácter literal al resultado, creando el formato visual.

Resultado del Proceso:

- Usuario escribe: 3519133949
- Máscara MX: '(XXX) XXX XXXX'
- Resultado final: '(351) 913 3949'

6. SIMULACIÓN ASÍNCRONA DE SERVIDOR (PROMISES)

Declaración de Función Asíncrona

```
function simularVerificacionServidor(email) {
   return new Promise((resolve) => {
      setTimeout(() => {
         const existe = correosRegistrados.includes(email.toLowerCase());
         resolve(existe);
      }, 1000);
   });
}
```

Conceptos Clave de Asincronía

Token/Concepto	Explicación Técnica
Promise	Objeto nativo de JS para manejar operaciones asíncronas. Representa un valor que puede estar disponible ahora, en el futuro, o nunca.
new Promise(executor)	Constructor. Recibe una función "ejecutora" que se llama inmediatamente con dos argumentos: resolve y reject.
(resolve) => {...}	Función ejecutora. resolve es una función que, al llamarse, marca la Promise como "cumplida" (éxito).
setTimeout(callback, delay)	Función nativa. Programa la ejecución de callback después de delay milisegundos. Simula latencia de red.
1000	Delay en milisegundos. 1000ms = 1 segundo.
.includes(valor)	Método de Array. Retorna true si valor existe en el array, false si no.
resolve(existe)	Cumple la Promise, pasando el valor booleano existe al siguiente .then() o await.

Consumo con async/await

```
formulario.addEventListener('submit', async (e) => {
   // ... validaciones previas ...

   const existeCorreo = await simularVerificacionServidor(inputEmail.value);

   if (existeCorreo) {
      // Manejar error de correo duplicado
   }
});
```

Token	Explicación
async	Palabra reservada. Marca la función como asíncrona, permitiendo el uso de await en su interior.
await	Pausa la ejecución de la función **hasta que la Promise se resuelva**. No bloquea el hilo principal del navegador (non-blocking).
const existeCorreo = ...	Cuando la Promise se resuelve, await retorna el valor pasado a resolve(), que se asigna a la variable.

Ventaja: El código se lee como síncrono (lineal), pero se ejecuta de forma asíncrona, mejorando legibilidad vs. el uso de .then().

7. VALIDACIÓN FINAL Y ENVÍO DEL FORMULARIO

Prevención de Comportamiento por Defecto

```
e.preventDefault();
```

- Cancela la acción por defecto del evento submit (recargar la página y enviar datos al servidor).
- Permite manejar el envío completamente con JavaScript (AJAX/fetch).

Patrón de Bandera de Validación

```
let esValido = true;

if (condicionFallida) {
   mostrarError(...);
   esValido = false;
}

if (!esValido) {
   return; // Detiene ejecución si hay algún error
}
```

- esValido: Variable bandera (flag) booleana.
- Cada validación que falla establece esValido = false.
- Al final, una sola condición if (!esValido) decide si continuar o abortar.

Validación de Email con Regex

```
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!emailRegex.test(inputEmail.value)) { ... }
```

Componente Regex	Significado
^	Ancla de inicio de string.
[^\s@]+	Uno o más caracteres que NO sean espacio ni @ (parte local del email).
@	Literal arroba (separador obligatorio).
\.	Punto literal (el . debe escaparse porque es metacarácter "cualquier carácter").
[^\s@]+	Uno o más caracteres para el dominio.
$	Ancla de fin de string.
.test(string)	Método de RegExp. Retorna true si el patrón coincide con todo el string.

Validación de Longitud de Teléfono

```
const telefonoLimpio = inputTelefono.value.replace(/[^0-9]/g, '');
if (telefonoLimpio.length !== config.longitud) { ... }
```

- Primero extrae solo dígitos (elimina máscara visual).
- Compara la longitud real con la requerida por la configuración del país.
- !==: Igualdad estricta. Compara valor **y** tipo (evita coerción implícita).

8. FUNCIONES AUXILIARES DE UI

Función mostrarError

```
function mostrarError(idError, elemento, mensaje) {
   const spanError = document.getElementById(idError);
   spanError.textContent = mensaje;
   elemento.classList.add('error');
   if(elemento.id === 'telefono') {
      elemento.parentElement.classList.add('error');
   }
}
```

Método/Propiedad	Propósito
textContent	Establece el contenido textual del elemento. Más seguro que innerHTML (evita inyección XSS).
classList.add('error')	Agrega la clase CSS 'error' al elemento. Activa estilos visuales definidos en styles.css (borde rojo, fondo rosado, etc.).
elemento.parentElement	Propiedad que retorna el nodo padre en el DOM. Usado para aplicar estilo al contenedor del teléfono (que incluye el prefijo).

Función limpiarError

```
function limpiarError(idError, elemento) {
   const spanError = document.getElementById(idError);
   spanError.textContent = '';
   elemento.classList.remove('error');
   if(elemento.id === 'telefono') {
      elemento.parentElement.classList.remove('error');
   }
}
```

- classList.remove('error'): Elimina la clase CSS, restaurando el estilo normal del elemento.
- textContent = '': Limpia el mensaje de error visual.

9. REFERENCIA RÁPIDA DE TOKENS JAVASCRIPT

Palabras Reservadas Clave

Token	Categoría	Uso en Este Script
const	Declaración	Variables de referencia inmutable (DOM elements, config).
let	Declaración	Variables cuyo valor cambiará (contadores, valores procesados).
function	Declaración	Definir funciones reutilizables (mostrarError, simularVerificacionServidor).
return	Control de Flujo	Terminar función y/o devolver un valor.
if / else	Control de Flujo	Ejecución condicional basada en validaciones.
for	Iteración	Bucle clásico para recorrer la máscara de teléfono.
async / await	Asincronía	Manejo elegante de operaciones asíncronas (simulación de servidor).
new	Operador	Instanciar objetos desde constructores (new Promise).

Operadores Comunes

Operador	Nombre	Ejemplo	Resultado
=	Asignación	x = 5	Guarda 5 en x
	Igualdad Estricta	5 === '5'	false (diferente tipo)
!==	Desigualdad Estricta	5 !== '5'	true
&&	AND Lógico	a > 0 && a < 10	true solo si ambas verdaderas
l l	OR Lógico	x === '' \l\l x === null	true si alguna es verdadera
!	NOT Lógico	!esValido	Invierte booleano
+=	Asignación con Concatenación	texto += 'más'	Equivale a texto = texto + 'más'

Métodos de String Útiles

Método	Propósito	Ejemplo
.toUpperCase()	Convertir a mayúsculas	'hola'.toUpperCase() → 'HOLA'
.toLowerCase()	Convertir a minúsculas	'HOLA'.toLowerCase() → 'hola'
.replace(regex, str)	Reemplazar coincidencias	'abc123'.replace(/[^0-9]/g, '') → '123'
.substring(inicio, fin)	Extraer porción	'telefono'.substring(0, 4) → 'tele'
.length	Obtener longitud	'hola'.length → 4

Métodos de Array Útiles

Método	Propósito	Ejemplo
.forEach(fn)	Ejecutar fn por cada elemento	[1,2,3].forEach(x => console.log(x))
.includes(valor)	Verificar existencia	['a','b'].includes('a') → true

Métodos de RegExp

Método	Propósito	Ejemplo
.test(string)	Verificar coincidencia	/^[0-9]+$/.test('123') → true

Métodos del DOM

Método	Propósito	Ejemplo
getElementById(id)	Buscar por ID	document.getElementById('miForm')
addEventListener(event, fn)	Registrar evento	btn.addEventListener('click', handler)
classList.add/claseList.remove	Manipular clases CSS	input.classList.add('error')
preventDefault()	Cancelar acción por defecto	e.preventDefault() en submit

CONSIDERACIONES FINALES DE SEGURIDAD Y RENDIMIENTO

1. **Validación del Lado del Cliente ≠ Seguridad**: Este script mejora la experiencia de usuario, pero **nunca** reemplaza la validación en el servidor. Un usuario malintencionado puede omitir JavaScript.

2. **Expresiones Regulares**: Las regex usadas son seguras para los casos de este formulario. Evita regex complejas con retroceso exponencial en producción.

3. **Acceso al DOM**: Seleccionar elementos una vez y guardar referencias (const inputX = ...) es más eficiente que buscar repetidamente con getElementById.

4. **Eventos input vs change**: input es más responsivo para validación en tiempo real; change es mejor para cuando el usuario termina de interactuar (ej: select).

5. **Async/Await**: Más legible que cadenas de .then(), pero recuerda manejar errores con try/catch en producción.

6. **Template Literals**: `error-${id}` es más legible y seguro que concatenación con +, especialmente con variables complejas.

Fin del Documento Técnico.